

Desain Sistem Aplikasi Solar Dryer IoT

1. Gambaran Umum Sistem

Sistem **Solar Dryer IoT** adalah platform monitoring dan kontrol pengering surya berbasis Internet of Things (IoT). Sistem ini memungkinkan pengguna memantau suhu & kelembaban dari **10 sensor DHT22** secara real-time, serta mengontrol kecepatan **blower BTS7960** melalui dashboard web maupun mobile.

Komponen Utama

Komponen	Teknologi	Fungsi
Firmware	ESP32 + Arduino C++	Membaca sensor, kontrol blower, kirim data
Backend/Database	Firebase Realtime Database	Penyimpanan & sinkronisasi data real-time
Autentikasi	Firebase Authentication	Login, session, RBAC
Web Dashboard	React 18 + Recharts	Monitoring & kontrol via browser
Mobile App (React)	React + Capacitor 8	APK Android dari codebase React
Mobile App (Flutter)	Flutter + fl_chart	Aplikasi mobile native Android
Hosting Web	Netlify	Deploy dashboard web

2. Arsitektur Sistem (High-Level)

```

graph TB
    subgraph HARDWARE["⚡ Hardware Layer"]
        DHT["10x DHT22  
Sensor Suhu & Kelembaban"]
        ESP["ESP32 DevKit V1  
(Mikrokontroler)"]
        BTS["BTS7960  
Motor Driver"]
        BLOWER["Blower AC"]
        RTC["RTC DS3231  
Real-Time Clock"]
        SD["SD Card Module  
Data Logger"]
    end

    subgraph CLOUD["☁ Cloud Layer (Firebase)"]
        AUTH["Firebase Auth  
Email/Password"]
        RTDB["Firebase RTDB  
asia-southeast1"]
    end

    subgraph CLIENT["📱 Client Layer"]
        WEB["React Web Dashboard  
Netlify"]
        APK_R["Android APK  
(React + Capacitor)"]
        APK_F["Flutter Mobile App  
(Native Android)"]
    end

    DHT -->|"GPIO 32,33,25,27,26,  
14,17,13,16,4"| ESP
    RTC -->|"I2C (SDA:21, SCL:22)"| ESP
    SD -->|"SPI (CS:5, SCK:18,  
MISO:19, MOSI:23)"| ESP
    ESP -->|"PWM GPIO 2  
Enable GPIO 15"| BTS
    BTS --> BLOWER
    ESP -->|"WiFi → HTTPS"| RTDB
    ESP -->|"Auth Token"| AUTH
    AUTH --> RTDB
    RTDB <-->|"onValue listener"| WEB
    RTDB <-->|"onValue listener"| APK_R
    RTDB <-->|"onValue stream"| APK_F

```

3. Arsitektur Hardware

3.1 Wiring Diagram

Komponen	Pin Komponen	Pin ESP32	Protokol
DHT22 Sensor 1-10	DATA	GPIO 32, 33, 25, 27, 26, 14, 17, 13, 16, 4	Digital
BTS7960 RPWM	RPWM	GPIO 2	PWM (5kHz, 8-bit)
BTS7960 Enable	R_EN & L_EN	GPIO 15	Digital
SD Card	CS	GPIO 5	SPI
SD Card	SCK	GPIO 18	SPI
SD Card	MISO	GPIO 19	SPI
SD Card	MOSI	GPIO 23	SPI
RTC DS3231	SDA	GPIO 21	I2C
RTC DS3231	SCL	GPIO 22	I2C

3.2 Kontrol Blower (BTS7960 PWM)

Kecepatan (0-100%) → map ke PWM (0-255)

└─ speed > 0 → Enable HIGH + ledcWrite(pwmValue)

└─ speed == 0 → ledcWrite(0) + Enable LOW (hemat daya)

PWM Config: 5kHz, 8-bit resolution, LEDC channel 0

4. Firmware ESP32 — Alur Kerja

4.1 Sequence Diagram

```
sequenceDiagram
    participant S as 10x DHT22
    participant E as ESP32
    participant SD as SD Card
    participant RTC as RTC DS3231
    participant FB as Firebase RTDB
    participant D as Dashboard

    Note over E: SETUP (Boot)
    E->>S: Init 10 sensor DHT22
    E->>RTC: Init RTC DS3231 (I2C)
    E->>SD: Init SD Card (SPI)
    E->>FB: WiFi Connect → Firebase Auth
    E->>FB: Set initial blower speed = 0
    E->>FB: Set control_mode = "manual"
    E->>FB: beginStream("/blowers")

    loop Setiap 60 detik
        E->>RTC: Baca timestamp
        E->>S: Baca 10 sensor (suhu + kelembaban)
        E->>E: Hitung rata-rata (avg_t, avg_h)
        E->>FB: SET /sensor_data (JSON detail)
        E->>SD: Append ke /data.csv
        E->>FB: PUSH ke /history (dengan server timestamp)
    end

    D->>FB: User ubah speed blower
    FB-->>E: Stream callback /blowers/blower1/speed
    E->>E: setBlowerSpeed() → PWM output
```

4.2 Interval & Timing

Aksi	Interval	Keterangan
Baca Sensor	60.000 ms (1 menit)	Baca 10 sensor + kirim ke Firebase
Push History	60.000 ms (1 menit)	Simpan ke node <code>/history</code>
Log SD Card	Setiap menit unik	Cek <code>lastLoggedMinute</code> berbeda
WiFi Reconnect	15.000 ms	Auto-check jika WiFi terputus

4.3 Struktur Data yang Dikirim ke Firebase

```
{
  "sensor_data": {
    "avg_t": 32.5,
    "avg_h": 61.2,
    "tanggal": "2026-06-22",
    "waktu": "12:15:30",
    "sensor_data": [
      { "sensor": 1, "t": 33.1, "h": 58.4 },
      { "sensor": 2, "t": "null", "h": "null" },
      ...
      { "sensor": 10, "t": 31.8, "h": 63.0 }
    ]
  }
}
```

4.4 Format Log SD Card (CSV)

```
Tanggal,Waktu,T1,H1,T2,H2,...,T10,H10
2026-06-22,12:15:30,33.1,58.4,NAN,NAN,...,31.8,63.0
```

5. Firebase — Struktur Database

5.1 Schema Lengkap

```
graph LR
    ROOT["🔥 Firebase RTDB Root"]

    ROOT --> SD["/sensor_data"]
    ROOT --> BL["/blowers"]
    ROOT --> CM["/control_mode"]
    ROOT --> HI["/history"]
    ROOT --> US["/users"]
    ROOT --> UA["/user_akses"]
    ROOT --> PR["/presence"]

    SD --> SD1["avg_t, avg_h, tanggal, waktu, sensor_data[]"]
    BL --> BL1["/blower1 {name, speed}"]
    CM --> CM1["'manual' | 'auto'"]
    HI --> HI1["/-NxABC {avg_t, avg_h, sensor_data[], timestamp}"]
    US --> US1["/{uid} {email, displayName, lastLogin}"]
    UA --> UA1["/{uid} {role, updatedAt}"]
    PR --> PR1["/{uid} {email, displayName, lastSeen}"]
```

5.2 Node Detail

Node	Tipe	Ditulis oleh	Dibaca oleh	Keterangan
/sensor_data	Object	ESP32	Dashboard	Data real-time terbaru
/blowers/blower1	Object	Dashboard	ESP32 (stream)	{name, speed: 0-100}
/control_mode	String	Dashboard	Dashboard	"manual" atau "auto"
/history/{pushId}	Object	ESP32	Dashboard	Data historis + server timestamp
/users/{uid}	Object	Dashboard (login)	Admin	Profil user
/user_akses/{uid}	Object	Admin / self (pertama kali)	Admin / self	Role assignment
/presence/{uid}	Object	Dashboard (auto)	Admin	Online/offline tracking

5.3 Security Rules

```
{
  "sensor_data": { "read": true, "write": true },
  "blowers":      { "read": true, "write": true },
  "control_mode": { "read": true, "write": true },
  "history":      { "read": true, "write": true },
  "users": {
    "read": "auth != null && role === 'admin'",
    "$uid": { "read/write": "auth.uid === $uid" }
  },
  "user_akses": {
    "read": "auth != null && role === 'admin'",
    "$uid": { "write": "admin ATAU belum ada data (self-init)" }
  }
}
```

6. React Web Dashboard — Arsitektur Frontend

6.1 Component Tree

```
graph TD
    IDX["index.js  
(Entry Point)"]
    AP["AuthProvider  
(Context)"]
    APP["App.jsx  
(Main Router)"]

    IDX --> AP
    AP --> APP

    APP --> LP["LoginPage"]
    APP --> HD["Header  
(Brand + Status + Logout)"]
    APP --> NAV["Tab Navigation"]
    APP --> TAB1["Tab: Dashboard"]
    APP --> TAB2["Tab: Control"]
    APP --> TAB3["Tab: Data"]
    APP --> TAB4["Tab: Management"]

    TAB1 --> RT["Real-Time Info Bar"]
    TAB1 --> SC["SensorCard x2  
(Avg Temp + Humidity)"]
    TAB1 --> ISC["IndividualSensorCard x10"]
    TAB1 --> CH["SensorChart  
(Recharts LineChart)"]

    TAB2 --> SS["Status Summary"]
    TAB2 --> FG["Fan Status Grid"]
    TAB2 --> CP["Control Panel  
(Auto/Manual Mode)"]
    CP --> BC["BlowerControl  
(PWM Slider)"]

    TAB3 --> DT["Data Table  
(Paginated, Searchable)"]
    TAB3 --> EX["Export CSV"]

    TAB4 --> MP["ManagementPage  
(User RBAC Table)"]
```

6.2 File & Tanggung Jawab

File	Ukuran	Fungsi
index.js	343 B	Entry point, mount <code>AuthProvider</code> + <code>App</code>
firebase.js	937 B	Inisialisasi Firebase App, export <code>database</code> & <code>auth</code>
AuthContext.jsx	5.9 KB	Auth state, RBAC, session timer (1 jam), presence
App.jsx	21.6 KB	Komponen utama: 4 tab, Firebase listeners, export CSV
SensorCard.jsx	2.8 KB	Kartu rata-rata + kartu individual per sensor
BlowerControl.jsx	2.5 KB	Slider PWM 0-100%, estimasi RPM (max 6000)
SensorChart.jsx	2.8 KB	Grafik line chart (Recharts) avg suhu & kelembaban
LoginPage.jsx	3.8 KB	Halaman login dengan error handling
ManagementPage.jsx	8.7 KB	Manajemen user: ubah role, hapus akses, lihat online
parseCsv.js	1.9 KB	Parser CSV dummy untuk mode development

6.3 Sistem RBAC (Role-Based Access Control)

```
graph LR
    subgraph ROLES["🔒 5 Level Role"]
        R0["guess  
Level 0"]
        R1["view_data  
Level 1"]
        R2["eksekusi_data  
Level 2"]
        R3["report_data  
Level 3"]
        R4["admin  
Level 4"]
    end

    subgraph TABS["📄 Tab Akses"]
        T1["Dashboard ✅"]
        T2["Control ⚙️"]
        T3["Data 📊"]
        T4["Management 👤"]
        T5["Export 📤"]
    end

    R0 --> T1
    R1 --> T1
    R1 --> T3
    R2 --> T1
    R2 --> T2
    R2 --> T3
    R3 --> T1
    R3 --> T3
    R3 --> T5
    R4 --> T1
    R4 --> T2
    R4 --> T3
    R4 --> T4
    R4 --> T5
```

Role	Level	Akses Tab	Export?
guess	0	Dashboard	✗
view_data	1	Dashboard, Data	✗
eksekusi_data	2	Dashboard, Control, Data	✗
report_data	3	Dashboard, Data	✅
admin	4	Dashboard, Control, Data, Management	✅

6.4 Session Management

- **Durasi:** 1 jam (3.600.000 ms)
- **Mekanisme:** Timestamp disimpan di `localStorage` → timer `setTimeout` otomatis logout
- **Presence:** `/presence/{uid}` di-set saat login, `onDisconnect().remove()` otomatis saat offline

6.5 Data Flow — Real-Time Listeners

```
sequenceDiagram
    participant FB as Firebase RTDB
    participant APP as App.jsx
    participant UI as UI Components

    APP->>FB: onValue("/sensor_data")
    FB-->>APP: {avg_t, avg_h, sensor_data[], waktu, tanggal}
    APP->>UI: setSensorData() → SensorCard, IndividualSensorCard

    APP->>FB: onValue("/blowers")
    FB-->>APP: {blower1: {name, speed}, blower2: {...}}
    APP->>UI: setBlowers() → BlowerControl, Fan Status

    APP->>FB: onValue("/history")
    FB-->>APP: [{avg_t, avg_h, sensor_data[], timestamp}, ...]
    APP->>UI: setHistory() → SensorChart, Data Table

    APP->>FB: onValue("/control_mode")
    FB-->>APP: "manual" | "auto"
    APP->>UI: setControlMode() → Mode Switch
```

6.6 Fitur Export CSV

- Format: `history_sensor_YYYY-MM-DD.csv`
- Kolom: `Tanggal, Waktu, Avg T, Avg H, S1-T, S1-H, ..., S10-T, S10-H`
- **Web:** Blob download via `<a>` tag
- **Android (Capacitor):** `Filesystem.writeFile()` → `Share.share()` (native share dialog)

7. Flutter Mobile App — Arsitektur

7.1 Struktur File

```
flutter_dashboard/lib/
├─ main.dart                # Entry point, MaterialApp (Dark Theme)
├─ screens/
│   └─ dashboard_screen.dart # Layar utama (StatefulWidget)
├─ widgets/
│   ├── sensor_card.dart    # Kartu suhu/kelembaban + status
│   ├── blower_control.dart # Slider PWM + RPM display
│   └─ chart_widget.dart    # Line chart (fl_chart)
└─ services/
    └─ firebase_service.dart # Abstraksi Firebase RTDB streams
```

7.2 Perbedaan Flutter vs React Dashboard

Aspek	React Dashboard	Flutter Dashboard
Multi-sensor	✔ 10 sensor individual	✘ Hanya avg (temperature/humidity)
Autentikasi	✔ Firebase Auth + RBAC	✘ Tidak ada login
Tab Navigation	✔ 4 tab (Dashboard, Control, Data, Management)	✘ Single page
Data Table	✔ Paginated + search + export	✘ Tidak ada
Theme	Light mode	Dark mode (Material 3)
Charting	Recharts (LineChart)	fl_chart (LineChart)
Max RPM Display	6000	2480
Presence/Session	✔	✘

7.3 Dependencies

Package	Versi	Fungsi
<code>firebase_core</code>	<code>^3.8.1</code>	Inisialisasi Firebase
<code>firebase_database</code>	<code>^11.2.1</code>	Realtime Database streams
<code>fl_chart</code>	<code>^0.69.2</code>	Grafik line chart
<code>google_fonts</code>	<code>^6.2.1</code>	Font Inter
<code>fluttertoast</code>	<code>^8.2.8</code>	Notifikasi toast

8. Capacitor — React → Android APK

8.1 Konfigurasi

```
// capacitor.config.ts
{
  appId: 'com.solardryer.iot',
  appName: 'Solar Dryer IoT',
  webDir: 'build',           // Output dari react-scripts build
  server: {
    url: 'https://solardryeriot.netlify.app', // Load dari Netlify
    androidScheme: 'https'
  }
}
```

8.2 Plugin Capacitor yang Digunakan

Plugin	Versi	Fungsi
<code>@capacitor/core</code>	<code>^8.2.0</code>	Core runtime
<code>@capacitor/filesystem</code>	<code>^8.1.2</code>	Tulis file CSV ke cache
<code>@capacitor/share</code>	<code>^8.0.1</code>	Native share dialog untuk export
<code>@capacitor/android</code>	<code>^8.2.0</code>	Android platform

9. Deployment & Distribusi

9.1 Platform Deployment

```
graph LR
    SRC["React Source Code"]
    BUILD["npm run build"]
    NET["Netlify"]
    solardryeriot.netlify.app
    CAP["Capacitor Sync"]
    GRADLE["Gradle Build"]
    APK_R["APK (React)"]
    FLUTTER["Flutter Build"]
    APK_F["APK (Flutter)"]

    SRC --> BUILD --> NET
    BUILD --> CAP --> GRADLE --> APK_R
    SRC --> FLUTTER --> APK_F
```

9.2 APK yang Dihasilkan

File APK	Ukuran	Keterangan
SolarDryerIoT-Auth-RBAC.apk	9.5 MB	Versi terbaru dengan autentikasi + RBAC
SolarDryerIoT-LiveUpdate.apk	11.5 MB	Versi live update dari Netlify
SolarDryerIoT_Flutter_PWM.apk	45 MB	Versi Flutter dengan kontrol PWM
SolarDryerIoT_React_PWM.apk	4.9 MB	Versi React ringan dengan PWM

10. Alur Data End-to-End

sequenceDiagram

participant S as 10x DHT22

participant E as ESP32

participant SD as SD Card

participant FB as Firebase

participant W as Web Dashboard

participant M as Mobile App

participant U as User

Note over S,U: === ALUR MONITORING (Setiap 1 Menit) ===

S->>E: Baca suhu & kelembaban (10 sensor)

E->>E: Hitung rata-rata, filter NaN

E->>FB: SET /sensor_data (JSON detail)

E->>FB: PUSH /history (+ server timestamp)

E->>SD: Append CSV row

FB-->>W: onDataChange → update real-time

FB-->>M: Stream → update real-time

W->>U: Tampilkan dashboard

M->>U: Tampilkan dashboard

Note over S,U: === ALUR KONTROL BLOWER ===

U->>W: Geser slider kecepatan (0-100%)

W->>FB: SET /blowers/blower1/speed = 75

FB-->>E: Stream callback → fanStreamCallback()

E->>E: setBlowerSpeed(0, 75)

E->>E: PWM = map(75, 0, 100, 0, 255) = 191

E->>E: Enable HIGH + ledcWrite(191)

Note over S,U: === ALUR LOGIN & RBAC ===

U->>W: Masukkan email + password

W->>FB: signInWithEmailAndPassword()

FB-->>W: Auth token

W->>FB: SET /users/{uid} (profil)

W->>FB: GET /user_akses/{uid} (role)

W->>W: Filter tab berdasarkan role

W->>FB: SET /presence/{uid} (online)

11. Konektivitas & Error Handling

11.1 Deteksi Online/Offline

Layer	Mekanisme	Timeout
ESP32 → WiFi	<code>WiFi.status()</code> check setiap 15 detik	Auto reconnect
ESP32 → Firebase	<code>Firebase.ready()</code> check sebelum write	Token auto-refresh
Dashboard	Bandingkan <code>lastUpdatedAt</code> vs <code>Date.now()</code>	120 detik (2 menit)

11.2 Error Handling pada ESP32

- Sensor gagal → tampilkan "GAGAL" di Serial, kirim `"null"` ke Firebase
- SD Card gagal → lanjut tanpa logging lokal (`sdReady = false`)
- WiFi gagal → retry otomatis di background
- Firebase timeout → retry pada iterasi loop berikutnya

12. Teknologi & Library

12.1 React Dashboard

Library	Versi	Fungsi
React	18.3.1	UI framework
Firebase SDK	10.14.1	Auth + Realtime Database
Recharts	2.13.3	Line chart visualization
react-icons	5.3.0	Icon set (Ionicons, Weather, FontAwesome)
react-hot-toast	2.4.1	Toast notifications
react-scripts	5.0.1	Build toolchain (CRA)

12.2 ESP32 Firmware

Library	Fungsi
Firebase_ESP_Client (Mobizt)	Firebase Auth + RTDB
DHT (Adafruit)	Baca sensor DHT22
RTCLib	RTC DS3231 (I2C)
SD + SPI	Data logging ke SD Card
WiFi	Koneksi jaringan

13. Ringkasan Desain

Sistem Solar Dryer IoT merupakan arsitektur **3-tier** yang terdiri dari:

1. **Hardware Tier** — ESP32 sebagai edge computing device yang membaca 10 sensor, mengontrol blower via PWM, menyimpan log ke SD card, dan berkomunikasi dua arah dengan Firebase.
2. **Cloud Tier** — Firebase sebagai backbone yang menyediakan real-time database (sinkronisasi data sensor + perintah kontrol), autentikasi (email/password), dan otorisasi (RBAC 5 level).
3. **Client Tier** — Dashboard multi-platform (React Web + Capacitor Android + Flutter Android) yang menyajikan monitoring real-time, kontrol blower, visualisasi grafik, tabel data historis dengan export CSV, dan manajemen user.

Seluruh komunikasi bersifat **event-driven** — ESP32 menggunakan Firebase stream listener untuk menerima perintah kontrol secara instan, sementara dashboard menggunakan `onVaLue` listener untuk menerima update data sensor secara real-time tanpa polling.